**WAB**

Provadis School of International Management and Technology

# Examining the Feasibility of Machine Learning-Based LogP Prediction on an ESP32 Microcontroller

A Proof-of-Concept Implementation and Performance Analysis

Rubin Chempananickal James

rubin.chempananickal-james@stud-provadis-hochschule.de

Matriculation Number: D876

Department: Information Technology

Module: Embedded Systems and Software

Reviewer: Prof. Dr. Eric Hutter

March 31, 2026

# Abstract

## English

The octanol-water partition coefficient (logP) is a key property in drug discovery and is often predicted using computational methods. This project investigates whether a quantized machine learning model can accurately predict logP values from ECFP4 fingerprints when deployed on an ESP32 microcontroller. A neural network (multi-layer perceptron) was trained on a large dataset of drug-like compounds, quantized, and deployed to the ESP32. Performance was evaluated on an independent 10% test subset. The model achieved a mean absolute error of 0.29, RMSE of 0.39, and $R^2$ of 0.92. Inference time was 53 ms per compound. The results demonstrate that real-time, on-device logP prediction is feasible on embedded hardware, though generalization to non-drug-like molecules remains a limitation.

## Deutsch

Der Oktanol-Wasser-Verteilungskoeffizient (logP) ist eine wichtige Eigenschaft in der Arzneimittelforschung und wird häufig mithilfe von Rechenmethoden vorhergesagt. In diesem Projekt wird untersucht, ob ein quantisiertes Machine-Learning-Modell logP-Werte aus ECFP4-Fingerprints vorhersagen kann, wenn es auf einem ESP32-Mikrocontroller ausgerollt wird. Ein neuronales Netzwerk (Multi-Layer-Perzeptron) wurde auf einem großen Datensatz arzneimittelähnlicher Moleküle trainiert, quantisiert und auf den ESP32 draufgespielt. Die Leistung wurde an einem unabhängigen 10%-Testdatensatz bewertet. Das Modell erreichte einen MAE von 0,29, einen RMSE von 0,39 und ein $R^2$ von 0,92. Die Inferenzzeit betrug 53 ms pro Molekül. Die Ergebnisse zeigen, dass eine Echtzeit-Vorhersage von logP-Werten auf eingebetteter Hardware möglich ist. Die Generalisierbarkeit auf nicht-arzneimittelähnliche Moleküle bleibt jedoch eine Einschränkung.

# Contents

# List of Figures

# List of Tables

# Glossary

**Adaptive Moment Estimation (Adam)** An optimization algorithm used in training machine learning models that computes adaptive learning rates for each parameter. 4

**ESP32** Espressif Systems' low-cost, low-power system on a chip (SoC) with integrated Wi-Fi and Bluetooth capabilities. 3, 5, 9, 10, II

**Extreme Gradient Boosting (XGBoost)** An ensemble learning method that uses gradient boosting to combine the predictions of multiple weak learners (usually decision trees) to create a stronger predictive model. 3

**Fingerprint** An algorithmically constructed binary vector representation of a molecule, where each bit represents the presence or absence of a particular substructure or feature in the molecule. 2, 3

**Hydrophilicity** The chemical property of being water-soluble, characterized by a highly negative logP value. 9, 10

**Lipophilicity** The chemical property of being fat-soluble, characterized by a highly positive logP value. 1, 10

**LogP** Logarithm of the octanol-water partition coefficient. A measure of a compound's hydrophobicity, or how well it dissolves in fats versus water. 1–4, 6–10, IV

**Multi-layer Perceptron (MLP)** A feed-forward artificial neural network composed of an input layer, one or more fully connected hidden layers, and an output layer. 4, 9

**QR code** Quick Response Code, a type of matrix barcode (or two-dimensional barcode) that can store information such as a URL, text, or other data. 2

**Random Forest (RF)** An ensemble learning method for classification and regression tasks that constructs multiple bootstrapped decision trees during training. 3

**Rectified Linear Unit (ReLU)** A type of activation function used in neural networks that outputs the input directly if it is positive, otherwise, it outputs zero. 4

**Simplified Molecular Input Line Entry System (SMILES)** A notation that allows a user to represent a chemical structure as a string of ASCII characters. 2–4, IV

**TensorFlow Lite (TFLite)** A framework for deploying machine learning models on mobile and embedded devices. Currently being rebranded as LiteRT. 3, 5

**ZINC20** ZINC Is Not Commercial (2020), a free database of commercially available compounds for virtual screening. 3, 4, 10

# 1 Introduction

## 1.1 Background

The octanol-water partition coefficient (logP) measures how well a compound dissolves in octanol versus water. It is a fundamental physicochemical property that quantifies the lipophilicity of a molecule.

> The logarithm of partition coefficient (logP) between n-octanol and water is a critical property related to the physicochemical and physiological properties of a drug molecule, such as absorption, distribution, metabolism, excretion and toxicity (ADMET) and target protein binding[1–4]. Thus it is frequently used in describing the druggability of a pharmaceutical molecule. The first logP calculation method was developed by Fujita et al. in 1960s[5,6]. Early on, the structural information of receptor is not easy to obtain, medicinal chemists frequently used quantitative structure-activity relationship (QSAR) to correlate the binding energy of a ligand with its physicochemical characteristics, and logP was regarded as a key descriptor measuring a drug's hydrophobicity. Nowadays, Lipinski's rule of five[7,8], published in the 1990s, has been widely used as simple rules to determine the druglikeness of a pharmaceutical molecule. logP smaller than 5 is one of the four rules in Lipinski's rule of 5.[9]

## 1.2 Motivation

To date, there have been many computational methods developed to predict logP values, including aggregating the logp contributions of individual fragments[10,11], free energy based calculations[9], and more recently, employing machine learning based methods to predict logP values based on a precomputed set of input features[12–14] .

Fragment based logP calculations involve having a lookup table of individual fragments, and a method of constructing a molecular graph and finding the fragments within it. Both this and the free energy based calculations require a non-trivial amount of computational resources, and are therefore infeasible to be implemented as a near real-time prediction method on a microcontroller.

The machine learning based approaches could potentially be feasible, if a neural network could be trained to predict logP values and translated into a microcontroller compatible format using LiteRT[15] (formerly TensorFlow Lite).

The reason for exploring the ability to predict logP on a microcontroller is to enable real-time, on-site predictions in various applications, such as environmental monitoring or portable drug discovery tools. By offloading the prediction task away from a host computer and onto a microcontroller, one can potentially reduce latency in systems that require immediate response, such as a microfluidic dispensing device that needs to make decisions based on the logP of a compound in real-time.

Since the molecular fingerprint is a bit vector that could be encoded as a QR code (see Figure 1.1 for an example), the microcontroller could also be used in scenarios where a user can scan said code printed on the sample in a laboratory or field setting with a portable device and receive an immediate prediction of the logP value, and if this approach seems feasible and accurate enough, it could be extended to predicting other properties as well, without needing to access a cloud-based service or a powerful computer.

Figure 1.1: A QR Code encoding the ECFP4 fingerprint of aspirin. The SMILES string was initially canonicalized via RDKit[16], and the fingerprint was generated using the Morgan algorithm[17] with a radius of 2 and a bit vector length of 2048. The resulting bit vector was converted to a byte stream then encoded into a QR code using the qrcode library in Python.

Also, by decoupling the fingerprint generation (a deterministic process on canonical SMILES strings) and the prediction (a non-deterministic process owing to the nature of machine learning models), and by binding the prediction to a specific piece of hardware (a "black box", as it were), one can make the entire system more modular and easier to debug. As an added bonus, the host computer wouldn't need to have any machine learning libraries installed.

## 1.3 Research Question

The main research question of this project is: How accurately can a quantized machine learning model predict logP values based on its ECFP4 fingerprint when deployed on an ESP32 microcontroller?

## 1.4 Previous Work

To date, there have been multiple efforts to train machine learning models to serve as logP predictors, such as the work by Ognichenko et al.[12] who trained an RF model on the 2D Simplex Representation of Molecular Structures (SiRMS) descriptors[18] of 10973 compounds, and the work by Baikété et al.[13] who trained a variety of machine learning models (RF, XGBoost, among others) on 623 molecular descriptors generated by the RDKit[16] and Mordred[19] packages for a dataset of 1,117 compounds. As of the time of writing, there have been no known efforts to deploy a machine learning based logP predictor on a microcontroller.

This current project aims to build upon the efforts of Gómez-Bombarelli et al.[20], who created a neural network based logP predictor on a subset of the ZINC20 dataset[21] as part of creating a variational autoencoder for molecular generation. Their dataset serves as the basis for training the model in this project, as it provides a large enough set of drug-like compounds with valid SMILES strings. This dataset was also chosen because it was used to train a neural network based logP predictor, which is the type of model that can be deployed on a microcontroller using LiteRT (formerly TFLite)[15].

# 2 Methods

## 2.1 Dataset

The base dataset used in this project is a subset of the ZINC20 dataset[21], which contains 249,455 commercially available drug-like molecules and their corresponding logP values. Specifically, this subset was extracted at random from the entire ZINC20 dataset by Gómez-Bombarelli et al.[20].

The logP values within the ZINC20 dataset were calculated using Molinspiration's MiLogP algorithm, which calculates logP based on group contributions[22]. Due to the limited availability of large datasets of experimentally confirmed logP values for drug-like molecules, and due to the limited scope of this project, this computed logP will be treated as the ground truth for the purposes of training and evaluating the machine learning model.

## 2.2 Design

Python[23] version 3.12.13 served as the primary programming language for this project. The entire project was conducted on a laptop running Windows 11 25H2.

The dataset was randomly split into training (80%), validation (10%), and test (10%) sets, with the random seed set to 42 to ensure reproducibility.

The input features for the model are 2048-bit Morgan fingerprints with radius 2 (also known as Extended-Connectivity Fingerprints diameter 4 (ECFP4)) generated from the SMILES representations of molecules. The SMILES were canonicalized and the fingerprint bit vectors were generated using the RDKit[16] library in Python.

The machine learning model is a MLP with a single hidden layer of 128 units, using a ReLU activation function, followed by a linear output layer that predicts the logP value. Keras[24] was used as the high-level API for building and training the neural network, while TensorFlow[25] served as the backend. Model training was performed fully on the CPU using the Adam optimizer[26] and mean squared error (MSE) as the loss function.

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{2.1}$$

where $y_i$ is the true logP value and $\hat{y}_i$ is the predicted logP value for the $i$-th sample.

Early stopping was employed to prevent overfitting, monitoring the validation loss and restoring the best weights. This meant that with a batch size of 256 and the TensorFlow random seed set to 42, the training converged after 12 out of a maximum of 20 epochs (see Figure 2.1).

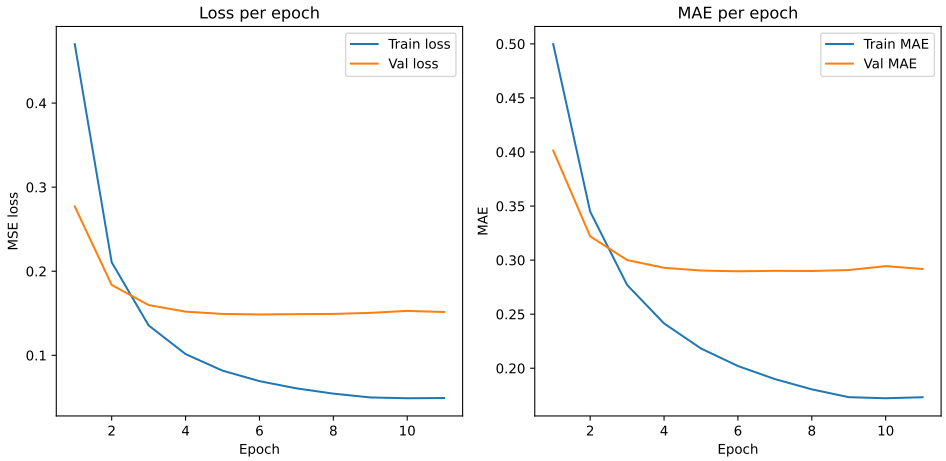


Figure 2.1: Training and validation loss and MAE curves over the 12 epochs.

## 2.3 ESP32 Deployment

To reduce the model size, the final model was quantized to 8-bit integers using TFLite's post-training quantization method. The resulting flatbuffer was saved as a .tflite file.

This .tflite file was then converted to a C array using a python tool that emulates the POSIX 'xxd' command, and the resulting .cc and .h files, along with a C++ API script, were built and flashed onto an ESP32 microcontroller using ESP-IDF version 5.5.3.

All the necessary code files, along with Visual Studio Code specific instructions for setting up the ESP-IDF environment, building the project, and flashing the firmware, can be found in the supplementary materials.

ESP32 specifications[27]:

- Model: ESP32-WROOM-32D
- CPU: Dual-core Xtensa LX6 microprocessor, up to 240 MHz
- RAM: 520 KB SRAM
- Flash Storage: 4 MB

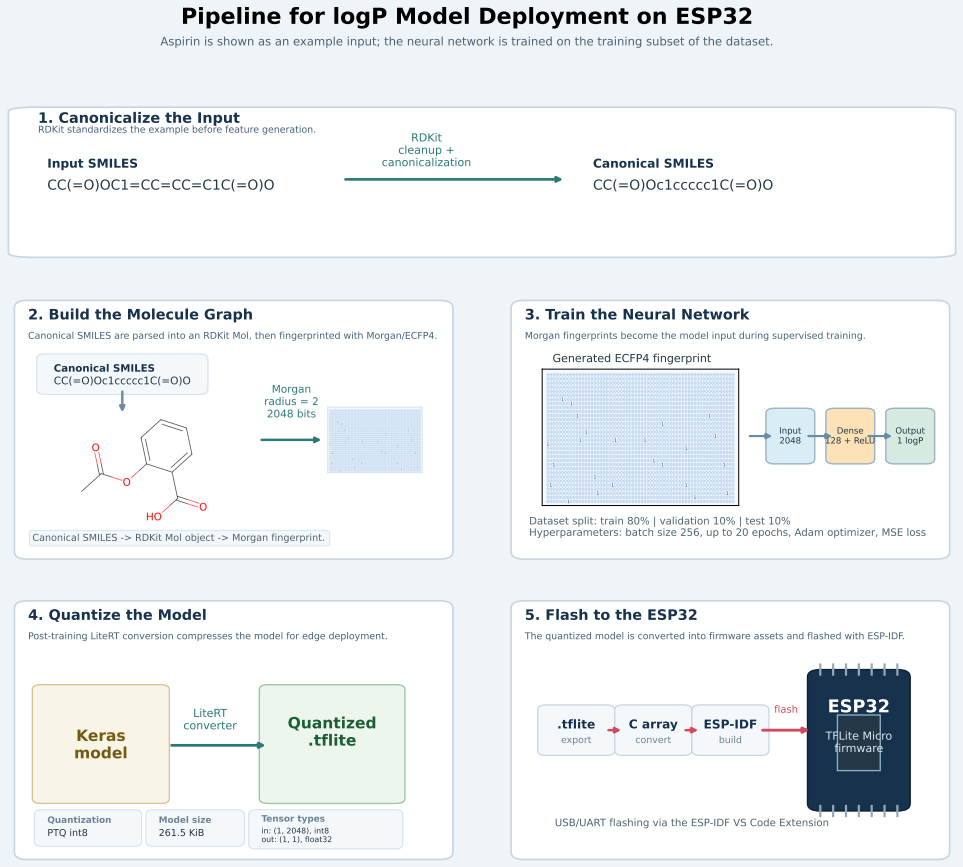


Figure 2.2: Diagram showing the stages of canonicalization, fingerprint generation, neural network training, quantization, and ESP32 deployment.

The accuracy of the model was then tested on the test set of 24,946 compounds by running inference on the ESP32 and comparing the predicted logP values against the ground truth. The measured performance metrics include mean absolute error (MAE), root mean squared error (RMSE), the coefficient of determination ($R^2$), and the Pearson correlation coefficient ($r$)[28].

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{2.2}$$

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{2.3}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} \tag{2.4}$$

$$r = \frac{\sum_{i=1}^{n}(y_i - \bar{y})(\hat{y}_i - \bar{\hat{y}})}{\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2 \sum_{i=1}^{n}(\hat{y}_i - \bar{\hat{y}})^2}} \tag{2.5}$$

where $n$ is the number of samples, $y_i$ is the true logP value, $\hat{y}_i$ is the predicted logP value, $\bar{y}$ is the mean of the true logP values, and $\bar{\hat{y}}$ is the mean of the predicted logP values.

# 3 Results

The trained model was successfully quantized and deployed on the ESP32 microcontroller, and inference was performed on the test set of 24,946 compounds. The performance metrics are summarized in Table 3.1, and the scatter plot of true vs. predicted logP values is shown in Figure 3.1. The distribution of absolute errors is depicted in Figure 3.2, and the molecule with the worst prediction error (citrate ion) is shown in Figure 3.3.

| Metric | Value |
|---|---|
| Number of Samples | 24,946 |
| Successfully Predicted | 100% |
| Mean Absolute Error (MAE) | 0.29 |
| Root Mean Squared Error (RMSE) | 0.39 |
| Maximum Absolute Error | 4.94 |
| $R^2$ | 0.92 |
| Pearson $r$ | 0.96 |
| Mean Inference Time | 53.13 ms |
| Mean Round Trip Time | 237.97 ms |

Table 3.1: Performance metrics on the test set.


Figure 3.1: Scatter plot of true vs. predicted logP values for the test set. The diagonal line represents perfect predictions. The "plateau" of predictions at the top right is likely due to a clamping effect caused by the quantization process.

| Metric | Value |
|---|---|
| Within 0.25 logP units | 53.12% |
| Within 0.50 logP units | 83.10% |
| Within 1.00 logP units | 98.06% |

Table 3.2: Percentage of predictions within certain absolute error thresholds.

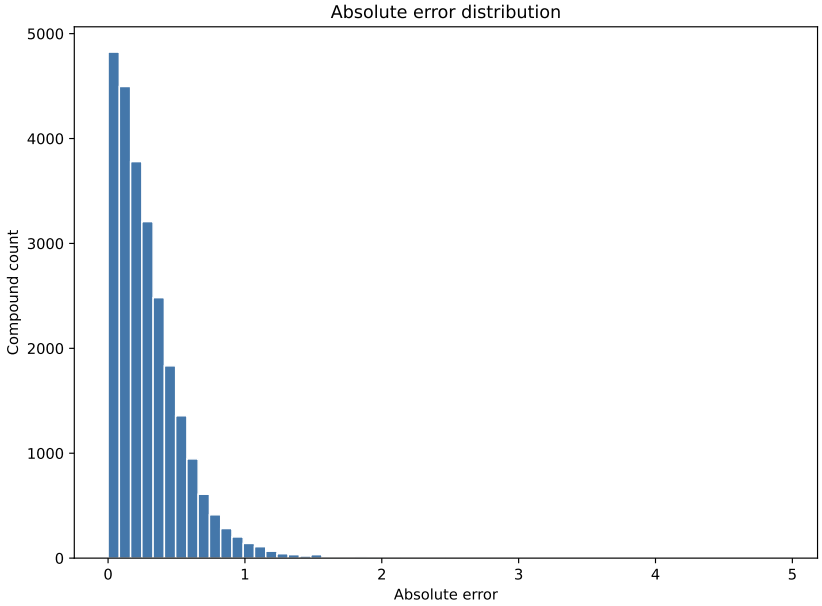


Figure 3.2: Histogram of absolute errors between predicted and true logP values. The majority of predictions have an absolute error below 0.5, with a long tail extending to higher errors.

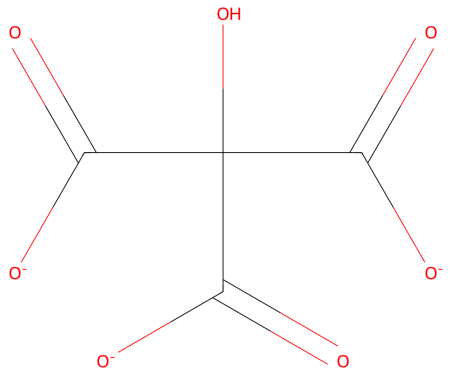


Figure 3.3: Citrate ion, the molecule with the worst prediction error (4.94). The ground truth logP is -6.03, while the predicted logP is -1.09.

# 4 Discussion

Out of the test set of 24,946 compounds, the ESP32-deployed model successfully predicted logP values of 100% of the samples. The perfect prediction rate is due to the fact that the dataset was well-curated and contained only valid SMILES strings that lent themselves to canonicalization and fingerprint generation without errors. Of note is the fact that the inference time is only 53.13 milliseconds per compound. This suggests that the reason the average round trip time took 237.97 milliseconds is probably not due to a bottleneck at the ESP32 level.

The high accuracy metrics reported in Table 3.1 would indicate that it is reasonable to assume that the logP values of a large number of drug-like compounds are highly correlated with their structural features, so much so that even a simple MLP with a single hidden layer can learn to extrapolate these values with high precision.

An interesting observation is the extremely inaccurate prediction of the logP value of citrate (IUPAC name: 2-hydroxypropane-1,2,3-tricarboxylate, pictured in Figure 3.3), whose true logP is -6.03 (extreme hydrophilicity) but the predicted logP was -1.09, resulting in an absolute error of **4.94**. This was by far the worst prediction in the entire test set, with the second worst only having an absolute error of **2.94** (See `Code/Results/worst_cases.csv` in the supplementary materials). What makes this case particularly noteworthy is that the predicted logP value is closer to that of citrate's neutral form citric acid, which has a reported logP of -1.72[29].

## 4.1 Limitations

The primary limitation of the model as it currently stands is that the highest logP value it can predict is 5.30, as can be seen in the scatter plot in Figure 3.1. This is likely a consequence of the quantization process, which clamps the output to the range of representable values in the int8 format. Even though there were compounds in the training set with logP values above 5.30, there were far too few of them (1384/224509, or 0.62%) for the TFLite quantization algorithm to learn to represent them accurately. The entire dataset comprised of drug-like molecules, and in general, the logP is unlikely to exceed this value according to Lipinski's rule of five[7]. So although this means the current model cannot predict the exact logP of highly lipophilic compounds, it can still be useful to recognize and flag such compounds, as they are unlikely to be orally bioavailable and would therefore make for poor drug candidates[8].

# 5 Conclusion

In conclusion, to answer the original research question, it seems that it is indeed possible to deploy a quantized machine learning model on an ESP32 microcontroller to predict logP values based on ECFP4 fingerprints with a high degree of accuracy. The proof-of-concept model achieved an MAE of 0.29, an RMSE of 0.39, and an $R^2$ value of 0.92 on the test set, which indicates that the predictions are closely aligned with the true values.

The inference time of 53.13 milliseconds per compound suggests that the model is efficient enough for real-time applications, although the overall round trip time of 237.97 milliseconds indicates that there may be some overhead in the communication stages that could be optimized in future iterations.

A notable shortcoming of the current model is that the dataset it was trained on was derived from the ZINC20 database of drug-like compounds, and therefore, the predictions may not generalize well to non-drug-like molecules or ones with extreme lipophilicity or hydrophilicity. So, a potential future direction for this research would be to train and test a model on a more diverse dataset of compounds with experimentally determined logP values.

# Bibliography


1. Hughes, J. D. *et al.* Physiochemical drug properties associated with in vivo toxicological outcomes. *Bioorganic & Medicinal Chemistry Letters* **18,** 4872–4875. ISSN: 0960-894X. https://www.sciencedirect.com/science/article/pii/S0960894X08008500 (2008).
2. Waring, M. J. Defining optimum lipophilicity and molecular weight ranges for drug candidates—molecular weight dependent lower log D limits based on permeability. *Bioorganic & Medicinal Chemistry Letters* **19,** 2844–2851 (2009).
3. Gleeson, M. P. Generation of a set of simple, interpretable ADMET rules of thumb. *Journal of Medicinal Chemistry* **51,** 817–834 (2008).
4. Johnson, T. W., Dress, K. R. & Edwards, M. Using the Golden Triangle to optimize clearance and oral absorption. *Bioorganic & Medicinal Chemistry Letters* **19,** 5560–5564 (2009).
5. Fujita, T., Iwasa, J. & Hansch, C. A new substituent constant, $\pi$, derived from partition coefficients. *Journal of the American Chemical Society* **86,** 5175–5180 (1964).
6. Iwasa, J., Fujita, T. & Hansch, C. Substituent constants for aliphatic functions obtained from partition coefficients. *Journal of Medicinal Chemistry* **8,** 150–153 (1965).
7. Lipinski, C., Lombardo, F, Dominy, B. & Feeney, P. Polar molecular surface properties predict the intestinal absorption of drugs in humans. *Adv. Drug Delivery Rev.* **46,** 3–26 (2001).
8. Lipinski, C. A. Lead-and drug-like compounds: the rule-of-five revolution. *Drug discovery today: Technologies* **1,** 337–341 (2004).
9. Sun, Y., Hou, T., He, X., Man, V. H. & Wang, J. Development and test of highly accurate endpoint free energy methods. 2: Prediction of logarithm of n-octanol-water partition coefficient (logP) for druglike molecules using MM-PBSA method. *Journal of Computational Chemistry* **44,** 1300–1311. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/jcc.27086. https://onlinelibrary.wiley.com/doi/abs/10.1002/jcc.27086 (2023).
10. Rekker, R. F. & Mannhold, R. Calculation of drug lipophilicity : the hydrophobic fragmental constant approach (1992).
11. Hansch, C., Rockwell, S. D., Jow, P. Y., Leo, A. & Steller, E. E. Substituent constants for correlation analysis. *Journal of medicinal chemistry* **20,** 304–306 (1977).

12. Ognichenko, L. N. *et al.* QSPR Prediction of Lipophilicity for Organic Compounds Using Random Forest Technique on the Basis of Simplex Representation of Molecular Structure. *Molecular Informatics* **31,** 273–280. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/minf.201100102. https://onlinelibrary.wiley.com/doi/abs/10.1002/minf.201100102 (2012).

13. Baikété, J., Malloum, A. & Conradie, J. Comparative study of machine learning methods for accurate prediction of logP and pKb. *Artificial Intelligence Chemistry* **4,** 100101. ISSN: 2949-7477. https://www.sciencedirect.com/science/article/pii/S2949747725000181 (2026).

14. Zang, Q. *et al.* In Silico Prediction of Physicochemical Properties of Environmental Chemicals Using Molecular Fingerprints and Machine Learning. *Journal of Chemical Information and Modeling* **57.** PMID: 28006899, 36–49. eprint: https://doi.org/10.1021/acs.jcim.6b00625. https://doi.org/10.1021/acs.jcim.6b00625 (2017).

15. *LiteRT is Google's on-device framework for high-performance ML & GenAI deployment on edge platforms* (visited on 29 March 2026). https://ai.google.dev/edge/litert.

16. *RDKit: Open-source cheminformatics software* (visited on 29 March 2026). https://www.rdkit.org/ (2026).

17. Morgan, H. L. The generation of a unique machine description for chemical structures-a technique developed at chemical abstracts service. *Journal of chemical documentation* **5,** 107–113 (1965).

18. Kuz'min, V. E. *et al.* Hierarchic system of QSAR models (1D–4D) on the base of simplex representation of molecular structure. *Journal of molecular modeling* **11,** 457–467 (2005).

19. Moriwaki, H., Tian, Y.-S., Kawashita, N. & Takagi, T. Mordred: a molecular descriptor calculator. *Journal of cheminformatics* **10,** 4 (2018).

20. Gómez-Bombarelli, R. *et al.* Automatic chemical design using a data-driven continuous representation of molecules. *ACS central science* **4,** 268–276. https://pubs.acs.org/doi/10.1021/acscentsci.7b00572 (2018).

21. Irwin, J. J., Sterling, T., Mysinger, M. M., Bolstad, E. S. & Coleman, R. G. ZINC: a free tool to discover chemistry for biology. *Journal of chemical information and modeling* **52,** 1757–1768 (2012).

22. *Molinspiration: logP - octanol-water partition coefficient* (visited on 29 March 2026). https://www.molinspiration.com/services/logp.html.

23. Van Rossum, G. & Drake, F. L. *Python 3 Reference Manual* ISBN: 1441412697 (CreateSpace, Scotts Valley, CA, 2009).

24. Chollet, F. *et al. Keras* (visited on 29 March 2026). 2015. https://keras.io.

25. Abadi, M. *et al. TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems* (visited on 29 March 2026). 2015. https://www.tensorflow.org/.

26. Kingma, D. & Ba, J. Adam: A Method for Stochastic Optimization. *International Conference on Learning Representations* (Dec. 2014).

27. *ESP32-WROOM-32D & ESP32-WROOM-32U Datasheet* (visited on 29 March 2026). https://documentation.espressif.com/esp32-wroom-32d_esp32-wroom-32u_datasheet_en.pdf (2026).

28. Pearson, K. Note on regression and inheritance in the case of two parents. *proceedings of the royal society of London* **58,** 240–242 (1895).

29. Książek, E. Citric Acid: Properties, Microbial Production, and Applications in Industries. *Molecules* **29.** ISSN: 1420-3049. https://www.mdpi.com/1420-3049/29/1/22 (2024).

# AI Declaration

The usage of AI tools within this project, including prompts used and outputs received, is documented here.

| System | Prompt | Usage |
|---|---|---|
| GitHub Copilot 1 | Asked to delete the existing project and provide a simple LaTeX template with a shared preamble, a Chapters subdirectory, TOC, glossary, abbreviations, and Roman numerals for non-content pages. | Template structure and LaTeX setup provided |
| GitHub Copilot 2 | Requested additional title page lines (WAB header, reviewer, module) for the main document. | Title page metadata and layout updated |
| GitHub Copilot 3 | Requested uppercase Roman numerals for front matter and lowercase Roman numerals for back matter page numbering. | Page numbering adjusted in main and exposee |
| GitHub Copilot 4 | Requested exposee title page to include shared info fields (WAB header, department, module, reviewer). | Exposee title page updated to include shared metadata |
| GitHub Copilot 5 | Requested adding the provadis-hochschule.pdf logo to the top right corner of both main and exposee title pages. | Logo added to top right of both title pages |
| GitHub Copilot 6 | Reported that Glossary and Abbreviations sections were missing from compiled output. | Added example `\newacronym` entries to abbreviations.tex so the Abbreviations section displays properly |
| GitHub Copilot 7 | Reported that glossary section still not appearing in final PDF. | Updated settings.tex to include `automake` option in glossaries package to enable automatic glossary generation |
| GitHub Copilot 8 | Asked for boilerplate to train a TFLite model for ESP32 to estimate logP from SMILES. | Provided a Python training script (RDKit + Keras) with TFLite export |
| GitHub Copilot 9 | Asked how to run inference on an ESP32 with the trained model. | Provided a high-level TFLite Micro deployment workflow for ESP32 |
| GitHub Copilot 10 | Asked for a beginner-friendly README in Code/ with step-by-step setup and usage. | Added a novice-oriented Code/README.md with installation, training, and ESP32 notes |

| System | Prompt | Usage |
|---|---|---|
| GitHub Copilot 11 | Asked for a TFLite Micro skeleton and beginner ESP32 setup steps. | Added Code/tflite_micro_skeleton.cpp and expanded Code/README.md with ESP32 instructions |
| GitHub Copilot 12 | Reported the TFLite Micro skeleton was incomplete. | Completed the skeleton with quantized support, Arduino setup, and a runnable loop |
| GitHub Copilot 13 | Asked to avoid Roman-numbered pages in glossary references. | Explained how `\glsaddall` affects page references |
| GitHub Copilot 14 | Asked to replace `iterrows` in the dataset loader with a better pattern. | Replaced loop with `Series.map` + boolean mask + `np.stack` for vectorized fingerprint loading |
| GitHub Copilot 15 | Asked for help to deploy the quantized logP model onto an ESP32 with LiteRT. | Added a dedicated ESP-IDF project, a host-side Python serial script, and usage instructions for the ESP32 inference workflow |
| GitHub Copilot 16 | Reported a serial port permission error while running the host SMILES sender against COM4. | Improved the Python script to catch serial open failures and print practical guidance about closing other serial monitors or processes |
| GitHub Copilot 17 | Asked for a test pipeline that runs the dataset against the ESP32 logP model and produces charts | Added a resumable Python evaluation pipeline with subset selection modes, CSV summaries, and PDF plots |
| GitHub Copilot 18 | Asked to reorganize the scripts for readability and maintainability. | Grouped imports, split constants into Paths and Hyperparameters sections, and reordered functions into Data, Model, Quantization and export, and Evaluation sections with separator comments |
| GitHub Copilot 19 | Asked the ESP32 evaluator to retry failed molecules up to 100 times instead of moving on, and to treat device errors and timeouts as non-terminal so reruns retry them. | Added MAX_RETRIES constant, retry loop with serial buffer flush and re-PING between attempts, and changed load_completed_indices to only skip ok, invalid_smiles, and max_retries_exceeded results |
| GitHub Copilot 20 | Reported that clamping persisted at 5.480893 after recalibration and asked how to eliminate it entirely. | Changed output type to float32 (TFLite inserts a DEQUANTIZE op; Dense layers still run int8) and updated the firmware to register AddDequantize, check for float32 output, and read data.f[0] directly. |

| System | Prompt | Usage |
|---|---|---|
| GitHub Copilot 21 | Reported a boot loop after reflashing with the float32 output model. | Diagnosed peak arena overflow from the DEQUANTIZE op scratch buffers overrunning the heap during Invoke(); doubled kTensorArenaSize from 64 KB to 128 KB. |
| GitHub Copilot 22 | Asked for a simple CLI script that takes a SMILES string and name, standardizes and canonicalizes it via the shared preprocessing module, generates a 2048-bit ECFP4 fingerprint, encodes the bit vector as a QR code, and saves it under the Results folder. | Added a Python CLI utility that writes QR images to Code/Results/qr_codes and installed the qrcode dependency for the active environment. |
| GitHub Copilot 23 | Reported a QR generation failure and then asked to read the saved QR code back into a bit vector and throw an error if the round trip does not match. | Changed the QR payload to a compact hex-encoded fingerprint and added OpenCV-based QR decoding plus strict round-trip verification before reporting success. |
| GitHub Copilot 24 | Asked for code to save a PDF graph of training/validation loss and MAE per epoch in Results/figures/history.pdf. | Added matplotlib code to plot and save training history after model training. |
| GitHub Copilot 25 | Asked for a Python and RDKit figure showing an aspirin SMILES string being canonicalized, drawn as a molecule graph, passed through the neural-network training workflow, quantized, and packaged for ESP32 deployment. | Added a Python script that generates a figure and documented the canonicalization, training, quantization, and flashing stages. |
| GitHub Copilot 26 | Asked for a simple RDKit script named molecule_picture.py that renders a specified molecule and saves it under Code/Results/molecules/worst_case.png. | Added a standalone Python script that canonicalizes the provided SMILES, renders the molecule with RDKit, creates the output directory if needed, and saves the PNG image to the requested location. |

# Declaration of Authorship

I hereby confirm that I have personally and independently prepared the present work and have not used any sources or aids other than those specified. All passages taken verbatim or in substance from other sources are identified as such. The drawings, illustrations and tables in this work are created by me or have been provided with an appropriate source reference. This work has not been submitted by me to any other university in the same or similar form for the acquisition of an academic degree.

Frankfurt, March 31, 2026 .....

Rubin Chempananickal James